

BEACON LABS

CYBERSECURITY EDUCATION LABS

Evil Twin Attack Detection Lab

Module: Network Security



BOISE STATE UNIVERSITY

COLLEGE OF ENGINEERING

Department of Computer Science

Table of Contents

Overview	3
Virtual Machine Setup	3
Task Set 1: Creating the Network with Mininet-WiFi	4
Task 1.1: Simulating the Network Topology	4
Task Set 2: Evil Twin Attack	7
Task 2.1: Setting up the Evil Twin Attack	7
Task Set 3: Evil Scout	9
Task 3.1: Creating the Capture File	10
Task 3.2: Implementing Evil Scout	11
Task 3.3: Disconnecting the Client	13
Task Set 4: Adversarial Thinking for Games with Imperfect Information	14
Task 4.1: Adversarial Thinking	14
Submission	16
Summary	16
References	16

Overview

Objective

Develop a comprehensive understanding of the intricacies involved in executing an Evil Twin attack, as well as devising defensive strategies from the perspective of the legitimate access point (LAP).

Evil Twin Attacks

Evil Twin attacks refer to a class of cybersecurity attacks where an attacker creates a rogue WiFi network that mimics a legitimate one. These types of attacks are typically executed on public WiFi networks, which are prone to many threats due to their open nature. By replicating the name and MAC address of a legitimate WiFi network, the attacker deceives clients into connecting to their network instead. This allows the attacker to perform a man-in-the-middle attack, intercepting and potentially altering the client's communication.

Previously, the only defense against this was for clients to modify their devices or for the network admin(s) to take additional measures to prevent it. However, Shrivastava et al.^[1] proposed a simple and lightweight piece of code known as Evil Scout to prevent Evil Twin attacks on open WiFi networks.

In this lab, you will use a program called Mininet-WiFi which allows for the simulation of a WiFi topology to perform this attack on.

Resources

Paper that the attack and defense are based upon:

P. Shrivastava, M. S. Jamal and K. Kataoka, "EvilScout: Detection and mitigation of evil twin attack in SDN enabled WiFi", IEEE Transactions on Network and Service Management, vol. 17, no. 1, pp. 89-102, 2020.^[1]

Virtual Machine Setup

A premade virtual machine image will be used. It requires at least 15 GB of space, and it is recommended to allocate a minimum of 2 GB of RAM.

The virtual machine contains all the files necessary to run this lab.

By understanding how this attack is executed and how to prevent it, you will gain a deeper understanding of the security vulnerabilities of public WiFi and the importance of secure network practices.

For setup instructions, see the provided Setup document.

Task Set 1: Creating the Network with Mininet-WiFi

Objective

Simulate the network topology in which an Evil Twin attack will be performed in order to enhance comprehension of the attack dynamics and how it can be defended against.

Task 1.1: Simulating the Network Topology

In this task we'll simulate the network topology. On the provided virtual machine, there are two directories and files of interest:

1. 'mininet-wifi' directory under /home/mininet/Documents.
2. 'miniedit.py' in the /home/mininet/Documents/mininet-wifi/examples directory which has a rudimentary GUI for creating a network topology.

The network topology will consist of a controller, switch, host, and an access point for our WiFi network. We will then create three stations on the network, two of which will act as legitimate clients attempting to connect to the network, and the final one will be our attacker. Our access point will have two WLANs, one for clients to connect to, and one that will monitor the WiFi channels. There should be links only between the controller, switch, access point, and host.

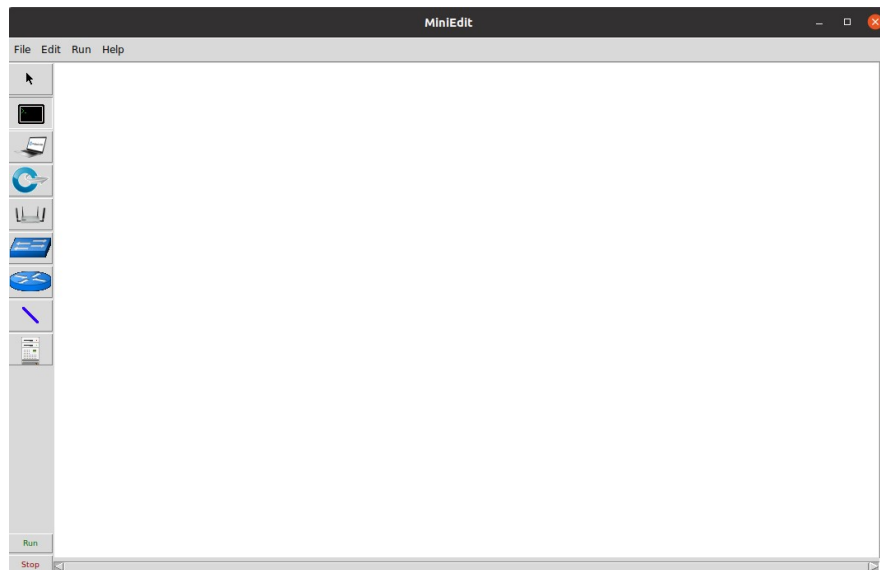


Figure 1: Miniedit's interface for creating the network topology

Action

- 1.1.1 - Run miniedit.py, and your interface should look like Figure 1 above.
- 1.1.2 - Create the topology as specified above. When it is complete, it should resemble Figure 2 on page 6 below.
- 1.1.3 - Alter the properties of the access point to have two WLANs and use channel six as seen in Figure 3 on page 6.

Save your topology as a '.mn' file so it can be altered with Miniedit in the future if needed. Additionally, export it as a level 2 script to create a Python file you can run.

Known Issues

Miniedit will not always create an access point with two WLANs even if specified. If this is the case, manually altering the python script will be needed.

Within your exported Python script, find where the specifications of the access point are set. It should be named 'ap1' and look like Figure 4 below. Add 'wlans=2' to this line in the constructor options if it is not there.

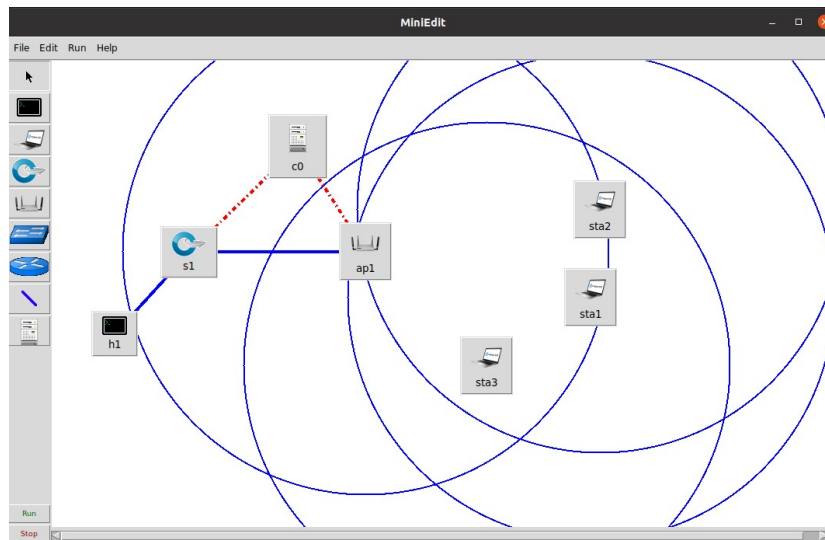


Figure 2: Completed topology in miniedit

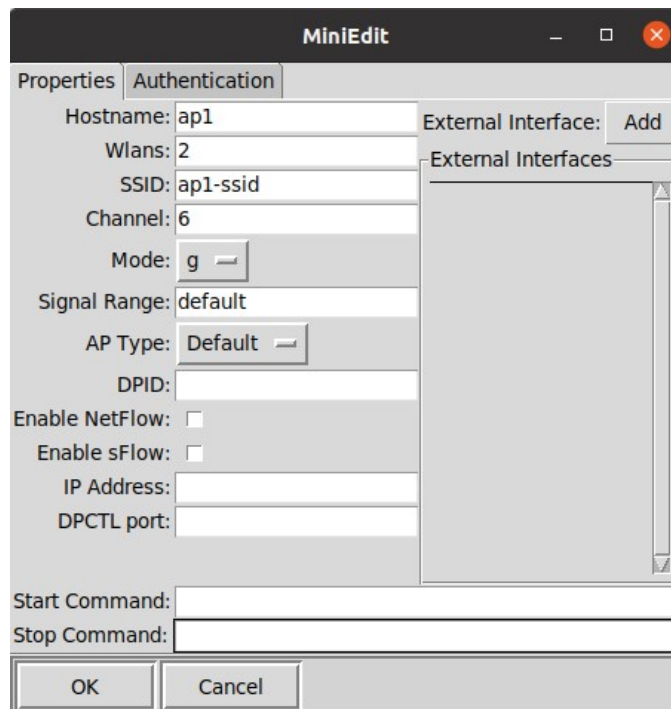


Figure 3: Right click on the access point to alter its properties

```
26
27     info( '*** Add switches/APs\n')
28     s1 = net.addSwitch('s1', cls=OVSKernelSwitch)
29     ap1 = net.addAccessPoint('ap1', cls=OVSKernelAP, ssid='ap1-ssid', wlans=2,
30                             channel='6', mode='g', position='409.0,234.0,0')
```

Figure 4: Add `wlans=2` to your python script if it is not included by exporting from `miniedit`

Task Set 2: Evil Twin Attack

Objective

Setup an Evil Twin attack via creation of a rogue WiFi access point that appears to be legitimate but, in reality, is controlled by the attacker.

Task 2.1: Setting up the Evil Twin Attack

The WiFi topology can now be run to set up the Evil Twin attack. The Python script exported in Task 1 will need to be run as an administrator (sudo). This will provide access to the Mininet-WiFi command-line interface. Below are some important commands that will be used within the mininet interactive terminal:

```
> nodes
Displays all nodes within the topology

> xterm <node>
Creates an external terminal to perform commands on a specific node

> py <node>.stop_()
Stops the specified node

> py <node>.start_()
Starts the specified node

> py <node>.setMasterMode(intf='<interface>', ssid='<desired_ssid>',
channel='<desired_channel>', mode='g')
Set the node's interface to master mode allowing clients to connect to
it. It allows you to specify the channel it runs on and the name of the
network (ssid)
```

Station 1 will be setup as the evil twin. Initially, the hardware address should be set to be the same as the LAP. By scanning for available networks, the LAP's name and hardware address can be located. To scan for nearby networks, use the command:

```
$ iwlist scan
```

The result should look like what's shown in Figure 5 on page 8 below.

```
sta1-wlan0 Scan completed :
  Cell 01 - Address: 02:00:00:00:03:00
            Channel:6
            Frequency:2.437 GHz (Channel 6)
            Quality=25/70  Signal level=-85 dBm
            Encryption key:off
            ESSID:"ap1-ssid"
            Bit Rates:1 Mb/s; 2 Mb/s; 5.5 Mb/s; 11 Mb/s; 6 Mb/s
                      9 Mb/s; 12 Mb/s; 18 Mb/s
            Bit Rates:24 Mb/s; 36 Mb/s; 48 Mb/s; 54 Mb/s
            Mode:Master
            Extra:tsf=0006014176647507
            Extra: Last beacon: 60ms ago
            IE: Unknown: 00086170312D73736964
            IE: Unknown: 010882848B960C121824
            IE: Unknown: 030106
            IE: Unknown: 2A0104
            IE: Unknown: 32043048606C
            IE: Unknown: 3B025100
            IE: Unknown: 7F080400000200000040
```

Figure 5: Result from scanning the network with iwlist

The next step involves setting this interface to master mode with the LAP's Extended Service Set ID (ESSID) and evaluating the difference between placing it on the same channel versus a different one.

Action

2.1.1 - Open an xterm terminal for Station 1 and scan for available networks.

2.1.2 - Using the `ifconfig` cli tool, alter the hw ether on Station 1's interface to match the LAP's hardware address, and then stop and restart the node.

2.1.3 - Set the interface to master mode with the LAP's ESSID.

2.1.4 - Run the command '\$ iw <interface> info' to verify if the hardware address, network name, and channel are correct, and the interface is running in master mode.

Finally, Station 1 will be the evil twin of the LAP. If one of the two stations is connected to the evil twin through scan, all packets should be sent to our attacker rather than the LAP.

Action

2.1.5 - Use one of the other stations to scan for networks and see what happens if the evil twin is running on the same channel vs a different channel and include the difference in your lab report.

2.1.6 - Set the evil twin (Station 1) to be on the same channel as the LAP and connect station 2 to it.

2.1.7 - Verify if the client is connected to the evil twin or the LAP by running 'tcpdump' on the access point and pinging a random IP address from the client. An easy way to tell is to also run tcpdump on the evil twin and see which one, if any, produces tcp traffic in the terminal.

If the client is connected to the LAP instead of the evil twin, disconnect the client using the command below. This command will later be used by the LAP to disconnect the client from the evil twin.

```
$ aireplay-ng -0 0 -a <mac_of_access_point> -c <_mac_of_client> <monitor_mode_interface>
```

Task Set 3: Evil Scout

Objective

Learn how to implement Evil Scout.

Evil Scout is a defensive mechanism designed to detect and neutralize the threat posed by an Evil Twin attack. It works by capturing and analyzing the packets being sent over a network to attempt to spot any anomalies or intrusion attempts.

Task 3.1: Creating the Capture File

With an evil twin capturing the client's packets on the network, it is now possible to implement Evil Scout to defend against it.

This involves capturing all packets that are sent over WiFi to the LAP's mac address then analyzing them to detect an evil twin. The program `airodump-ng` from the aircrack-ng software suite (preinstalled on the provided VM) will be used for this purpose. This program will also be used to facilitate the disconnection of the client from the evil twin, ensuring no harm can be done by their captured packets.

If the network was set up correctly, the LAP should have two WLANs. One is used for clients to access, and the second is used to monitor network traffic. Through the below command's interface, many specifications about the network can be seen.

```
$ airodump-ng <monitor_mode_interface>
```

Action

3.1.1 - Set the second WLAN on the access point to monitor mode instead of master mode.

3.1.2 - Run the 'airodump-ng' command above on the LAP. Your terminal should look like Figure 6 below.

3.1.3 - Try sending packets from the client to the evil twin.

3.1.4 - Note in your lab report which field(s) increase.

Once this is complete, a capture file can be created using the command below for use with evil scout.

```
$ airodump-ng --bssid <lap_mac_address> --channel <lap_channel> --write <filename> <monitor_mode_interface>
```

With the above command active, send a couple pings from the client to the evil twin so the capture file has some data. Once you have a couple of

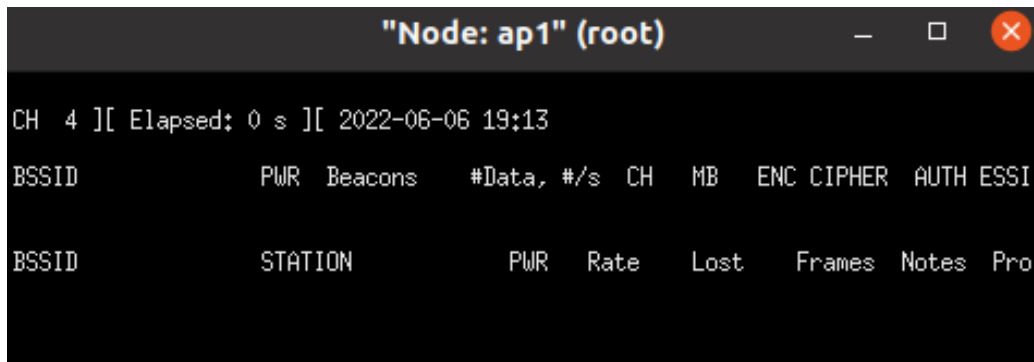


Figure 6: Airodump-ng's interface

packets, you can stop airodump-ng while focusing the terminal it's running in and using the key combo CTRL+C. You should then see a '.cap' file in the Documents directory with your specified filename.

Task 3.2: Implementing Evil Scout

Inside the Documents directory, there is a Python script titled 'EvilScout.py'. This program will implement two of the four cases that Evil Scout uses to detect the presence of evil twins.

In the first case, it checks to see if the IP prefix assigned to the client is the same prefix that is assigned to legitimate clients. In the second case, it checks the hardware source address to see if it matches any connected clients. Since only packets that intend to reach the LAP are captured, disconnecting clients connected to other LAPs poses no risk.

Action

3.2.1 - Complete EvilScout.py, using the code skeleton for the script (Figure 7 on page 12) provided in the Documents directory, and run it on the LAP. Include pictures of your completed EvilScout script and the result of running it on the LAP in your lab report.

For case two, a list of the current stations connected to the LAP must be created. The script 'station_list_testing.py' is included in the Documents directory to make this easier to test. It can also be seen in Figure 8 below.

```
1 #!/usr/bin/env python3
2 from scapy.all import *
3 import subprocess
4
5
6
7 filename = '' #define the capfile name here
8
9
10
11
12 def evilScout(pkt):
13     #setting variables
14
15     sourceIp = # get the packet source ip from the ARP layer
16     sourceMac = # get the packet source hardware address from the ARP layer
17
18     sourceIpPrefix = # use a substring to get the ip rprefix of the captured
19     assignedIpPrefix = "" # what ip prefix is being assigned by the LAP
20
21     #case 1
22     print("CASE 1")
23     print("Captured ip-prefix: " + sourceIpPrefix)
24     print("Expected ip-prefix: " + assignedIpPrefix)
25     if sourceIpPrefix != assignedIpPrefix: # if the captured prefix is
26         print("CASE 1 TRUE") # then case 1 is true
27         return True
28
29
30     #case 2
31     currentStationList = stationList() # Finish the method to produce the mac a
32     print("CASE 2")
33     print("captured MAC: " + sourceMac)
34     print("stations alloted: ")
35     print(*currentStationList, sep = ", ")
36     if not (sourceMac in currentStationList):
37         print("CASE 2 TRUE")
38         return True
```

Figure 7: Evil Scout code skeleton to be completed

The completed Evil Scout script should display a message along the lines of "Evil Twin Detected!" if there's an Evil Twin and "No Evil Twin Detected!" if there isn't one. When you are satisfied with your evil scout implementation, you can move on to Task 3.3.

```

1 #####
2 # Use this file to test getting a station list #
3 #####
4
5 from scapy.all import *
6 cmd = ['iw', 'dev', ' ', 'station', 'dump'] # Enter your LAP interface in the empty section
7
8 output = subprocess.Popen( cmd, stdout=subprocess.PIPE ).communicate()[0]
9
10 #decode the output from the subprocess to a string
11 outputString = output.decode()
12
13 #list of stations that we will return from this method
14 stationList = []
15
16 #keep track of where we are in the output
17 currentIndex = 0
18
19 # try printing the outputString to see what data it lists out
20 # this line can be commented out when you no longer need it
21 print(outputString)
22
23 # What should we look for in outputString to get only the mac addresses
24 query = ' '
25
26 while currentIndex != -1:
27     currentIndex = outputString.find(query, currentIndex)
28
29     #if we find one then add it to the list
30     if(currentIndex != -1):
31         stationList.append( ) # append the mac address to the stationList
32
33         #iterate the current index forward so we do not repeat stations
34         currentIndex =
35
36 print(*stationList, sep = ", ")
--

```

Figure 8: *station_list_testing.py* script to test acquisition of the LAP station list easily

Task 3.3: Disconnecting the Client

Aireplay-ng also allows for sending deauthentication packets to clients to make them disconnect from a network. It's essential to confirm that the client is connected to an evil twin and not the legitimate access point before sending a deauthentication. After the deauthentication, they can attempt to connect to the valid network.

Use the command below to send this packet.

```
# aireplay-ng -0 0 -a <mac_of_access_point> -c <mac_of_client> <monitor_mode_interface>
```

The first zero in the command is the flag used to indicate that this will be a deauthentication that is being sent. The second zero is to specify the number of packets that will be sent. Making this value 0 makes them send continuously.

Action

3.3.1 - Run `iwconfig` on the client and ensure that it is connected to the evil twin.

3.3.2 - Run the 'aireplay-ng' command above after and compare the difference.

3.3.3 - Include your observations in your lab report.

Task Set 4: Adversarial Thinking for Games with Imperfect Information

Objective

Gain an understanding of adversarial thinking by being presented a hypothetical cybersecurity situation that highlights the strategic choices and trade-offs faced by both the defender and attacker.

Task 4.1: Adversarial Thinking

Imagine a scenario where you setup a server that happens to be vulnerable to an attacker using an evil twin attack.

An attacker can choose to either attack the server or do nothing. If they attack, they can then choose to have the evil twin exist for a long time, defined as a long attack, or for a short amount of time, defined as a short attack.

As the server, you do not know whether the attacker is performing a long attack or a short attack, and you must decide whether to implement evil scout or do nothing. The advantage to doing nothing is that your users

would be unaware that there was a data breach and your company's stock price would not decrease, but you run the risk of sensitive data being stolen. The longer the attack, the more data has the potential to be lost. The attacker also wants the server to use more resources in defending the attack, so making them implement Evil Scout from a short attack benefits the attacker as well.

Let's look at the game theory table presented in Figure 9 below.

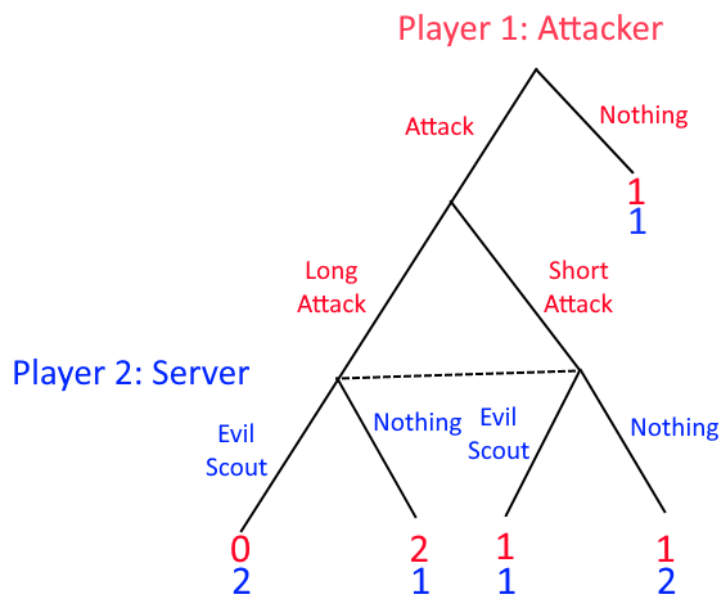


Figure 9: Game Theory table with imperfect information

Read, understand, and analyze the table to answer the following questions. Be sure to include the questions in your lab report. The dotted line represents that Player 2 does not know the action that Player 1 will take, which is called *imperfect information*. Both players can see the table and must play based on their assumptions of what the other player will do.

Action

- 4.1.1 - Should player one always choose to attack? Why or why not?
- 4.1.2 - When attacking, should a short or long attack be used? Explain.
- 4.1.3 - Knowing that the server is unaware of the attack being short or long, should player 2 implement evil scout or do nothing? Explain.

Submission

Submit a lab report using the provided [lab report template](#). Submit the lab report to your instructor with what you struggled with, what you learned, and images depicting the steps you took. Additionally, include your code for the evilScout.py python script. Include answers to the questions in task 4. Text and images are required for full points.

Summary

This lab explored the insecurity of public WiFi, a relatively simple way to set up an evil twin which can capture client packets, how a network can defend against this type of attack, and the implications of implementing this kind of defense from a Game Theory perspective.

The Evil Scout defense against Evil Twin attacks can be implemented in networks with just a software update and does not require any client-side work or additional hardware used in previous methods of defense. This allows for quick, easy, and efficient deployment of detection for Evil Twin attacks.

References

- [1] Pragati Shrivastava, Mohd Saalim Jamal, and Kotaro Kataoka. Evilsout: Detection and mitigation of evil twin attack in sdn enabled wifi. *IEEE Transactions on Network and Service Management*, 17(1):89–102, 2020.